[Copy]CM.EDITAR+ MVP Specification

CM.EDITAR+ Comprehensive MVP Specification and Agent Prompt

Purpose and Overview

Objective:

Build **CM.EDITAR+**, a desktop application that gives users full, safe, auditable control over the Windows Explorer **New** submenu and provides an optional per-user **New+** submenu for curated templated file types. The product must be Windows first and designed so the UI and core services can be reused on Linux and macOS in the future. This document is a complete, start-from-scratch specification and AI agent prompt intended to be fed into a capable code generation agent (Replit, Claude Code, or similar) to implement the application end to end.

Why this product is unique

No existing tool combines a safe, [per-user “ShellNew” discovery and editing, a template manager with placeholder resolution, a non-elevated file creation runtime, and an auditable snapshot/rollback system that enforces HKCU-only runtime changes and provides a “New+” per-user submenu that appears immediately after the default New menu.]

CM.EDITAR+ fills that gap!

Target audience

Power users, developers, system administrators, and teams who want safe, repeatable control over the New menu and templated file creation without risking system integrity.

High Level Architecture

Components

- **UI Application**
  - Desktop app built with Avalonia UI on .NET 7 or .NET 8 using MVVM.
  - Runs non-elevated by default.
  - Hosts Registry Editor, Template Manager, New+ installer UI, and Help walkthrough.
- **Registry Service**
  - Library that performs discovery, dry-run manifest generation, and HKCU writes.

    - Exposes JSON APIs for discovery and preflight.
- **Apply Service**
    - Responsible for snapshot creation, applying HKCU writes, calling SHChangeNotify, verification, and rollback.
- **FileCreator Runtime**
    - Non-elevated CLI executable that resolves templates and writes files atomically.
    - Accepts command line and named pipe RPC.
- **Installer Helper**
    - Elevated WiX custom action used only during install/uninstall.
    - Writes installer snapshots and optionally registers New+ parent keys.
- **IPC Layer**
    - Named pipes on Windows with ACL restricted to current user.
    - Unix domain sockets on Linux and macOS.
    - Per-install secret stored with DPAPI on Windows and OS keyring on other OSes.
- **Storage**
    - Templates: %AppData%\CM.EDITAR\Templates\<id>
    - Snapshots: %LocalAppData%\CM.EDITAR\Backups\<ISO-timestamp>.reg
    - Audit log: %LocalAppData%\CM.EDITAR\Audit\changes.log
    - Logs: %LocalAppData%\CM.EDITAR\Logs\<timestamp>.log
- **Testing and Automation**
    - Unit tests with xUnit or NUnit.
    - UI automation with Playwright for Windows or WinAppDriver.
    - Integration tests for MSI install and New+ end-to-end.

Data Flow

1. UI requests discovery from Registry Service.
2. Registry Service returns JSON manifest of ShellNew entries.
3. User stages edits in UI; UI shows dry run manifest.
4. On Apply, Apply Service creates snapshot, writes HKCU keys, calls SHChangeNotify, verifies, and logs.
5. FileCreator handles dynamic template creation via command or IPC.

Technology Choices and Rationale

Primary Stack

- **Language**: C# on .NET 7 or .NET 8
  - **Why**: Strong Windows integration, mature tooling, cross platform runtime, and excellent testability.
- **UI Framework**: Avalonia UI
  - **Why**: XAML style development, WPF parity on Windows, and cross platform support for Linux and macOS.
- **Installer**: WiX Toolset for MSI packaging
  - **Why**: Robust MSI authoring, custom actions, and enterprise compatibility.
- **IPC**: Named pipes on Windows, Unix domain sockets on Linux/macOS
  - **Why**: Native, secure local IPC with ACL support.
- **Secrets**: DPAPI on Windows, OS keyring on Linux/macOS
  - **Why**: Platform native secure storage.
- **Testing**: xUnit or NUnit, Playwright or WinAppDriver
  - **Why**: Reliable unit and UI automation frameworks.

Cross Platform Considerations

- Keep platform specific code isolated behind interfaces.
- Registry Service is Windows only; on Linux/macOS the discovery layer becomes a no-op or reads a local manifest for portable mode.
- FileCreator uses platform appropriate atomic write semantics and IPC.
- UI and Template Manager are cross platform via Avalonia.

Security and Safety Requirements

Non Destructive Policy

- **HKCU only at runtime**. No HKLM writes without explicit elevated installer action.
- **Snapshot before any write**: export affected keys to %LocalAppData%\CM.EDITAR\Backups\<ISO-timestamp>.reg.
- **Snapshots metadata**: include timestamp, user SID, list of keys exported, and SHA256 of the .reg file.
- **Dry run preview**: show exact registry keys and values to be changed.

- **Explicit confirmation**: require user confirmation for apply; require typed confirmation for high risk actions.
- **Automatic rollback**: if verification fails, restore snapshot and log the event.
- **Audit trail**: append signed changelog entry to %LocalAppData%\CM.EDITAR\Audit\changes.log.
- **No automatic deletions**: propose cleanup only; require human sign off.

IPC Security

- Named pipe ACL limited to current user.
- Per-install secret stored with DPAPI on Windows and OS keyring on other platforms.
- Client must present secret token to FileCreator server; server validates token and template id.
- Reject unknown template ids and untrusted commands.

Template Sanitization

- Templates are data only by default.
- Command templates require explicit user approval and show a security warning.
- Disallow templates that reference network paths or attempt to run installers without explicit consent.

Threat Model Summary

- Protect against malicious templates and unauthorized IPC access.
- Ensure backups are local and not uploaded automatically.
- Provide clear user consent for any elevated or system wide action.

Registry Model and ShellNew Details

ShellNew Types

- **NullFile**: creates an empty file.
- **FileName**: copies a static file from a path.
- **Command**: runs a command to create the file.
- **Data**: stores raw file bytes in the registry.

Discovery Rules

- Prefer HKCU\Software\Classes for reads and writes at runtime.

- Fallback to HKCR for discovery only (read-only).
- Discovery output JSON fields:
  - extension
  - displayName
  - shellNewType
  - fileName
  - command
  - data
  - visible
  - sourceKey
  - risk

Example Registry Preview

Preview only, not applied

```
Key: HKCU\Software\Classes\.md\ShellNew
Value: FileName (REG_SZ) =
C:\Users\Ethan\AppData\Roaming\CM.EDITAR\Templates\blank.md
```

Example .reg snippet to add FileName

```
Windows Registry Editor Version 5.00

[HKEY_CURRENT_USER\Software\Classes\.md\ShellNew]
"FileName"="C:\\Users\\Ethan\\AppData\\Roaming\\CM.EDITAR\\Templates\\blank.md"
```

Template Engine Specification

Template Metadata

- **Fields**
  - id UUID
  - name string
  - extensions array of strings
  - category string
  - defaultFilename string
  - templateType enum NullFile|FileName|Command|Data
  - templateSource string (file path or command)
  - placeholders array of {name, default, description}

- createdAt ISO timestamp
- modifiedAt ISO timestamp
- author string

Placeholder Engine

- Built in placeholders: %DATE%, %TIME%, %USERNAME%, %YEAR%, %GUID%
- Custom placeholders defined per template.
- Placeholder resolution occurs in FileCreator.
- Preview UI resolves placeholders with sample values.

Template Storage

- Each template stored in folder %AppData%\CM.EDITAR\Templates\<id>\
  - template.md or template.bin
  - metadata.json
- Templates are UTF-8 encoded for text templates.

Template Manager Features

- Create, edit, duplicate, delete templates.
- Map templates to extensions and categories.
- Preview resolved output.
- Export and import templates and packs.
- Validation and sanitization for Command templates.

FileCreator Runtime Specification

CLI Interface

- CM.EDITAR.FileCreator.exe --create --template-id <id> --target "<path>"
- CM.EDITAR.FileCreator.exe --preview --template-id <id> returns resolved preview to stdout.
- CM.EDITAR.FileCreator.exe --serve starts named pipe server for RPC.

Atomic Write Behavior

1. Resolve template and placeholders to a temporary file in same target directory.
2. Ensure file permissions and attributes are set.
3. Move temporary file to final target path using atomic rename.

4. Return success or detailed error.

IPC Behavior

- Server listens on named pipe with ACL restricted to current user.
- Client connects and sends JSON request with templateId, targetPath, and token.
- Server validates token against DPAPI stored secret and validates template id.
- Server performs create and returns JSON response with status and diagnostics.

Security

- Reject unknown template ids.
- Log operations to %LocalAppData%\CM.EDITAR\Logs.
- Rate limit or throttle requests to prevent abuse.

UI and UX Specification

Global Layout

- **Header**: App title, installed version label, Help dropdown, global search, elevation indicator.
- **Left panel**: Categories list with hover tooltips and quick filters.
- **Center**: Main table with extensions and bulk actions.
- **Right panel**: Selected entry card, Custom Add card, Runtime Status card.
- **Footer**: Apply Changes, Undo Last, Undo All, Flush Shell Cache, Preflight, Export manifest, Import manifest.

Main Table Columns

- **Queue**: staging order indicator
- **Ext**: file extension
- **New menu label**: display name in New menu
- **Group**: category or pack
- **State**: enabled, missing, or disabled
- **Risk**: rec, warn, high
- **Description**: short description

Selected Entry Card

- Preview registry target path.

- Quick toggles for enable/disable (staged).
- Edit template button opens Template Manager.
- Preview resolved template content.

Template Manager Window

- Template list with search and filters.
- Editor with placeholder insertion and live preview.
- Map to extensions UI.
- Export and import buttons.

Help and Walkthrough

- First run interactive walkthrough.
- Contextual tooltips on hover for every control.
- Full searchable help document with rollback instructions and examples.

Accessibility

- Keyboard navigation for all controls.
- Screen reader labels and ARIA roles.
- High contrast theme and scalable fonts.

Installer and Packaging

WiX MSI

- Per-user install by default.
- Installer tasks:
  - Copy executables and templates.
  - Create Start Menu and Desktop shortcuts.
  - Optionally register New+ parent keys in HKCU.
  - Create installer snapshot in Backups.
  - Add uninstall entry that attempts to restore installer snapshot.
- Uninstall behavior:
  - Attempt to restore installer snapshot.
  - Remove New+ keys created by installer.

- Leave user templates unless user opts to remove them.

Portable ZIP

- Extract to %LocalAppData%\CM.EDITAR.
- Register per-user ShellNew entries without elevation where possible.
- Document limitations and manual steps for users.

Code Signing

- Integrate signing step in local build scripts.
- CI placeholder for signing with secure key storage.

Testing Strategy

Unit Tests

- RegistryService discovery and dry run.
- Template serialization and placeholder resolution.
- FileCreator atomic write and validation.

Integration Tests

- MSI install/uninstall flows.
- End to end test: install, run app, Fix Integration, create file via Explorer New menu, verify content.

UI Automation

- Playwright or WinAppDriver scripts to simulate user flows: discovery, staging edits, preflight, apply, rollback.

Acceptance Tests

- All unit tests pass.
- Integration E2E test that creates a file via New+ and verifies content.
- Security audit checklist completed.

CI CD and Release

Local Build Scripts

- scripts/build.ps1 builds solution and runs tests.
- scripts/build-installer.ps1 builds WiX MSI.
- scripts/build-portable.ps1 builds portable ZIP.

CI Pipeline

- Build, run unit tests, run static analyzers, run security checks, produce artifacts, sign artifacts if keys available.

Release

- Publish signed MSI and portable ZIP.
- Attach release notes and audit log summary.

Roadmap and Milestones

Sprint 0 Safety and Discovery

- Discovery CLI that enumerates ShellNew entries (HKCU-first).
- Snapshot/rollback service.
- Dry run Registry Editor UI.

Sprint 1 Template Manager and FileCreator

- Template model and storage.
- Template Manager UI with preview.
- FileCreator CLI and named pipe skeleton.
- Unit tests for template serialization and placeholder resolution.

Sprint 2 New+ Integration and Installer

- New+ parent registration and ordering.
- WiX MSI and portable ZIP.
- Integration tests for New+ end to end.

Sprint 3 Polish and Security

- IPC auth with DPAPI secret.

- Security audit fixes.
- UI polish and accessibility pass.
- CI signing integration.

Sprint 4 Cross Platform Port

- Port UI to Linux and macOS via Avalonia.
- Replace registry discovery with local manifest for non Windows platforms.
- Packaging for other OSes.

Acceptance Criteria and Verification Commands

Acceptance Criteria

- Discovery returns accurate list for HKCU and HKCR fallback.
- Dry run preview shows exact registry keys and values.
- Snapshot created before write with metadata and SHA256.
- Apply writes only to HKCU and triggers SHChangeNotify.
- FileCreator resolves placeholders and writes atomically.
- New+ appears immediately after default New in Explorer.
- Uninstall restores snapshots created by installer.
- Unit and integration tests pass locally.

  \

Local Verification Commands

Build and tests

```
dotnet build CM.EDITAR.sln
dotnet test CM.EDITAR.sln
```

Discover current New menu

```
dotnet run --project src/ContextMenuManager -- discover-new > newmenu.json
type newmenu.json
```

Create template file

```
$templates = "$env:AppData\CM.EDITAR\Templates"
New-Item -ItemType Directory -Path $templates -Force
Set-Content -Path (Join-Path $templates "blank.md") -Value "# Notes" -Encoding UTF8
```

Create file from template

```
.\src\CM.EDITAR.FileCreator\bin\Debug\net6.0\CM.EDITAR.FileCreator.exe --create --template-id <id> --target "C:\Temp\test.md"
```

Export HKCU snapshot

```
$backup = "$env:LocalAppData\CM.EDITAR\Backups\classes-hkcu-$(Get-Date -Format o).reg"
New-Item -ItemType Directory -Path (Split-Path $backup) -Force
reg export "HKCU\Software\Classes" "$backup" /y
```

Restore snapshot

```
reg import "C:\Path\To\classes-hkcu-2026-05-11T18-00-00.reg"
```

Detailed Developer Tasks and Stories

Epic Discovery and Safety

- **Story**: Implement discovery CLI and JSON manifest.
  - **Tasks**: Read HKCU, fallback to HKCR read only, serialize to camelCase JSON.
  - **Tests**: Discovery unit tests.
- **Story**: Implement snapshot and rollback service.
  - **Tasks**: Export .reg, compute SHA256, write metadata JSON.
  - **Tests**: Snapshot and restore unit tests.

Epic Template Manager

- **Story**: Template metadata model and storage.
  - **Tasks**: Implement metadata schema, file layout, serialization.
  - **Tests**: Serialization tests.
- **Story**: Template Manager UI.
  - **Tasks**: List, create, edit, preview, map to extensions.
  - **Tests**: UI automation smoke tests.

Epic FileCreator

- **Story**: FileCreator CLI and atomic write.
  - **Tasks**: Implement placeholder resolution, atomic write, error handling.
  - **Tests**: Unit tests for placeholder resolution and atomic write.
- **Story**: Named pipe skeleton and DPAPI secret.
  - **Tasks**: Implement server and client skeleton, store secret with DPAPI.
  - **Tests**: IPC auth tests.

Epic New+ and Installer

- **Story**: New+ parent registration and ordering.

    - **Tasks**: Determine ordering technique and implement registration.
    - **Tests**: Integration test verifying New+ ordering.
- **Story**: WiX MSI packaging and uninstall restore.
    - **Tasks**: WiX authoring, custom actions for snapshot creation and restore.
    - **Tests**: MSI install/uninstall tests.

Example Agent Prompt for Full Build

Use this exact block as the instruction for an AI agent starting from scratch. Paste as a single message.

"CM.EDITAR+" AGENT BUILD PROMPT

Context
You are building "CM.EDITAR+" from scratch. The goal is a Windows-first desktop app that discovers and edits the Explorer New menu safely and installs an optional per-user New+ submenu. The app must be non-destructive, HKCU-only at runtime, snapshot-backed, auditable, and test-covered. The UI must be cross-platform ready via Avalonia and the FileCreator runtime must be non-elevated with secure IPC.

Deliverables
1) Discovery CLI that enumerates ShellNew entries and outputs JSON.
2) Snapshot and rollback service that exports .reg snapshots with metadata and SHA256.
3) Registry Editor UI with staged dry-run preview and apply flow.
4) Template Manager backend and UI with placeholder engine and preview.
5) FileCreator CLI and named pipe server with DPAPI per-install secret.
6) WiX MSI and portable ZIP packaging with installer snapshot and uninstall restore.
7) Unit tests, integration tests, and UI automation tests.
8) Documentation: user guide, rollback instructions, release notes.

Constraints
- No destructive actions without snapshot and human approval.
- All runtime writes must be HKCU only.
- Templates are data only; Command templates require explicit approval.
- IPC must be authenticated with per-install secret and named pipe ACL.

Return format
For each task return a JSON object:
{
"task": "<short task name>",
"status": "TODO|IN-PROGRESS|DONE",
"changed_files": ["path/to/file1", "path/to/file2"],
"tests_added": ["TestName1", "TestName2"],
"verification_command": "<single-line command to run locally>",
"notes": "<short human-readable notes or blockers>"
}

Start with Sprint 0 tasks:
- Implement discovery CLI and snapshot/rollback service.
- Implement Registry Editor UI skeleton with dry-run preview.
- Add unit tests for discovery and snapshot.

Stop and request human approval before any destructive action.

End of prompt.

Example .reg Revert Template

Only use after human approval and after verifying snapshot exists

```
Windows Registry Editor Version 5.00

; Revert CM.EDITAR added ShellNew for .md
[-HKEY_CURRENT_USER\Software\Classes\.md\ShellNew]

; If you prefer to restore previous values, import the snapshot .reg instead
```

Documentation and Support Plan

- **User guide**: Quick start, preflight, apply, restore, template manager tutorial.
- **Admin guide**: Installer options, per-machine vs per-user, enterprise deployment.
- **Troubleshooting**: How to restore snapshots, how to remove New+ manually, how to inspect logs.
- **Security report**: One page threat model and must-fix items.

Final Notes and Next Steps

Immediate next step for you

- Choose whether you want the agent to implement Sprint 0 (Discovery and Safety) or Sprint 1 (Template Manager and FileCreator). Paste the corresponding short agent prompt block into your chosen AI agent.

If you want me to produce additional artifacts

- I can generate ready-to-paste code skeletons, sample unit tests, WiX snippets, or a GitHub Actions pipeline YAML. Tell me which artifact you want first and I will produce it in Markdown.

Appendix A Example FileCreator Request JSON

```
{
"action": "create",
"templateId": "b3f9c2d4-8a2e-4f1a-9c2b-1a2b3c4d5e6f",
"targetPath": "C:\\Users\\Ethan\\Desktop\\NewDocument.md",
"token": "base64-token"
}
```

Appendix B Example Snapshot Metadata JSON

```
{
"snapshotId": "2026-05-11T18:00:00Z",
"userSid": "S-1-5-21-...",
"keysExported": [
"HKCU\\Software\\Classes\\.md\\ShellNew",
"HKCU\\Software\\Classes\\.json\\ShellNew"
],
"regFile": "classes-hkcu-2026-05-11T18-00-00.reg",
"sha256": "3a7bd3f4..."
}
```

This Markdown document is intentionally exhaustive and structured so you can paste it into any capable AI agent and get a consistent, complete build plan. If you want, I will now expand any single section into code skeletons, WiX snippets, unit test examples, or a step by step developer checklist. Which artifact should I produce next: code skeletons, WiX snippets, unit tests, or CI pipeline YAML?